Universidad El Bosque · Ingeniería de Sistemas

Bases de Datos 2 · Docente: Christian Felipe Duarte

Patrones de modelado de documentos en MongoDB

Los doce patrones del resumen «Building with Patterns», con ejemplos aplicados al proyecto SIRAV

Integrantes

David Felipe Bejarano Montes · José David Rodríguez

Entregable

Video de 12 a 13 minutos (YouTube, público)

Fecha

3 de noviembre de 2026

Contenido

Resumen
Los doce patrones
Guion del video
Entregable y lista de verificación

Resumen

El taller pide un video donde se explique cada patrón de modelado de documentos de MongoDB: qué es, cuándo conviene usarlo y un ejemplo concreto en JSON. La referencia es el resumen «Building with Patterns: A Summary» del blog de MongoDB, que repasa doce patrones. Esta página reúne el material que sirve de base para el video: la explicación de cada patrón, un ejemplo con datos de SIRAV (el sistema de reservas y alquiler de vehículos del proyecto final) y el guion con los tiempos.

En todos los patrones la idea de fondo es la misma. En MongoDB los datos se organizan según cómo los va a usar la aplicación, y no según una normalización previa. Por eso los patrones casi siempre cambian algo de duplicación o de complejidad en la aplicación por menos uniones, menos escrituras o consultas más sencillas.

> **Estado del video**
>
> El video todavía no está publicado. Cuando esté en YouTube con visibilidad pública, aquí irá el enlace directo.

#	Patrón	Para qué sirve
1	Approximation (aproximación)	Reducir escrituras cuando no hace falta precisión exacta
2	Attribute (atributo)	Buscar sobre campos parecidos con un solo índice
3	Bucket (cubeta)	Agrupar datos de flujo continuo por intervalo
4	Computed (calculado)	Guardar resultados ya calculados
5	Document Versioning (versionado de documentos)	Conservar el historial de un documento
6	Extended Reference (referencia extendida)	Evitar uniones copiando los datos más usados
7	Outlier (valor atípico)	Que los casos raros no condicionen el diseño
8	Pre-allocation (preasignación)	Crear la estructura completa desde el inicio
9	Polymorphic (polimórfico)	Guardar documentos parecidos en la misma colección
10	Schema Versioning (versionado de esquema)	Evolucionar el esquema sin detener el sistema
11	Subset (subconjunto)	Dejar en el documento solo lo que se usa más
12	Tree (árbol)	Modelar jerarquías

Los doce patrones

Cada ejemplo usa el contexto de SIRAV. Son propuestas de modelado para explicar el patrón, no el esquema final del proyecto.

1. Approximation (Aproximación)

Qué es. En lugar de guardar un valor exacto cada vez que cambia, se guarda un valor aproximado y se actualiza solo de vez en cuando. La lógica vive en la aplicación.

Cuándo usarlo. Cuando hay cálculos o escrituras muy frecuentes y la precisión exacta no es lo más importante, por ejemplo un contador de visitas.

Frente al modelo relacional. En un modelo relacional cada visita sería un UPDATE sobre la misma fila. Aquí se reduce el número de escrituras.

Ventajas y desventajas. A favor: menos escrituras y números estadísticamente válidos. En contra: el número no es exacto y la implementación queda en la aplicación.

Código P1 · Contador aproximado de vistas (mongosh)

```
// Contador de vistas de un vehículo: solo se escribe 1 de cada 100 visitas
if (Math.random() < 0.01) {
  db.vehiculos.updateOne(
    { _id: "VEH-0021" },
    { $inc: { vistas_aprox: 100 } }
  );
}
```

2. Attribute (Atributo)

Qué es. Cuando muchos documentos comparten campos parecidos que se quieren buscar u ordenar, esos campos se pasan a un array de pares clave-valor.

Cuándo usarlo. Cuando los documentos son grandes, tienen muchos campos similares y solo algunos documentos tienen cada campo.

Frente al modelo relacional. En relacional se resolvería con una tabla de atributos (modelo EAV) y varios JOIN. Aquí basta un solo índice sobre el array.

Ventajas y desventajas. A favor: menos índices y consultas más simples y rápidas.

Código P2 · Especificaciones de un vehículo como pares clave-valor

```
{
  "_id": "VEH-0021",
  "placa": "KLM123",
  "especificaciones": [
    { "k": "transmision", "v": "automática" },
    { "k": "combustible", "v": "híbrido" },
    { "k": "capacidad_maletero_l", "v": 420 }
  ]
}
// Un único índice cubre cualquier especificación:
db.vehiculos.createIndex({ "especificaciones.k": 1, "especificaciones.v": 1 })
```

3. Bucket (Cubeta)

Qué es. Los datos que llegan como flujo (lecturas, eventos) se agrupan en un documento por intervalo de tiempo en lugar de un documento por dato.

Cuándo usarlo. En series de tiempo, analítica en tiempo real e IoT.

Frente al modelo relacional. Un documento por lectura llenaría la colección de millones de filas pequeñas. Con cubetas hay menos documentos y los índices pesan menos.

Ventajas y desventajas. A favor: menos documentos, mejor rendimiento de índices y acceso simple gracias a los datos preagregados.

Código P3 · Lecturas GPS de una hora en un solo documento

```
{
  "vehiculo_id": "VEH-0021",
  "inicio": ISODate("2026-11-03T08:00:00Z"),
  "fin":    ISODate("2026-11-03T09:00:00Z"),
  "n_lecturas": 3,
  "velocidad_prom": 41.7,
  "lecturas": [
    { "t": ISODate("2026-11-03T08:00:10Z"), "lat": 4.7110, "lng": -74.0721, "vel": 38 },
    { "t": ISODate("2026-11-03T08:20:10Z"), "lat": 4.7231, "lng": -74.0402, "vel": 45 },
    { "t": ISODate("2026-11-03T08:40:10Z"), "lat": 4.7502, "lng": -74.0310, "vel": 42 }
  ]
}
```

4. Computed (Calculado)

Qué es. Los valores que la aplicación calcularía una y otra vez al leer se calculan una vez al escribir y se guardan en el documento.

Cuándo usarlo. En accesos de lectura muy intensivos donde se repite el mismo cálculo.

Frente al modelo relacional. En relacional se usaría un SUM/COUNT con GROUP BY en cada consulta. Aquí se lee el total ya listo.

Ventajas y desventajas. A favor: menos carga de CPU y consultas más simples. En contra: a veces es difícil detectar la necesidad y no conviene abusar.

Código P4 · Acumulados precalculados por vehículo

```
// Al cerrar una reserva se actualizan los acumulados del vehículo
db.vehiculos.updateOne(
  { _id: "VEH-0021" },
  { $inc: { num_reservas: 1, ingresos_totales: 280000 } }
);
```

5. Document Versioning (Versionado de documentos)

Qué es. Se guarda el documento actual en una colección y las versiones anteriores en otra, con un número de revisión.

Cuándo usarlo. Cuando hay que conservar el historial de cambios de un documento.

Frente al modelo relacional. En relacional se necesitarían tablas de auditoría con triggers. Aquí la versión vigente se consulta sin tocar el historial.

Ventajas y desventajas. A favor: fácil de implementar, incluso en sistemas ya existentes, y sin impacto en las consultas de la última revisión. En contra: duplica las escrituras y hay que consultar la colección correcta.

Código P5 · Tarifa vigente y su historial

```
// Colección tarifas (versión vigente)
{ "_id": "TAR-SUV", "categoria": "SUV", "precio_dia": 210000, "revision": 4 }

// Colección tarifas_historial (versiones anteriores)
{ "tarifa_id": "TAR-SUV", "categoria": "SUV", "precio_dia": 195000, "revision": 3,
  "vigente_hasta": ISODate("2026-09-30T00:00:00Z") }
```

6. Extended Reference (Referencia extendida)

Qué es. Se copia, dentro del documento que referencia, solo la información de otro documento que se consulta con frecuencia, para evitar el JOIN.

Cuándo usarlo. Cuando se hacen muchos JOIN para combinar datos que se piden juntos.

Frente al modelo relacional. En relacional una reserva necesitaría unirse a cliente y vehículo cada vez. Aquí los datos más usados ya están dentro de la reserva.

Ventajas y desventajas. A favor: lecturas más rápidas y menos uniones. En contra: se duplican datos.

Código P6 · Reserva con datos copiados de cliente y vehículo

```
{
  "_id": "RES-1042",
  "estado": "confirmada",
  "desde": ISODate("2026-11-10"), "hasta": ISODate("2026-11-13"),
  "cliente":  { "id": "CLI-88", "nombre": "Laura Gómez", "ciudad": "Bogotá" },
  "vehiculo": { "id": "VEH-0021", "placa": "KLM123", "modelo": "Mazda CX-30" }
}
```

7. Outlier (Valor atípico)

Qué es. Los pocos documentos que se salen de lo normal se tratan aparte para que no condicionen el diseño del caso típico.

Cuándo usarlo. Cuando unas pocas consultas o documentos no encajan con el resto y terminarían guiando toda la solución.

Frente al modelo relacional. En relacional habría que diseñar la tabla pensando en el caso extremo. Aquí el caso normal sigue siendo simple.

Ventajas y desventajas. A favor: los atípicos no dictan la solución y los casos típicos quedan a la medida. En contra: está pensado para consultas concretas y las ad hoc pueden rendir mal; buena parte se implementa en la aplicación.

Código P7 · Marca de desbordamiento para un caso atípico

```
// Vehículo normal: últimas reservas embebidas
{ "_id": "VEH-0021", "ultimas_reservas": ["RES-1042", "RES-1050"], "desbordado": false }

// Vehículo de flota muy usado: se marca y el resto va a otra colección
{ "_id": "VEH-0007", "ultimas_reservas": ["RES-2210", "RES-2214"], "desbordado": true }
// reservas_extra: { "vehiculo_id": "VEH-0007", "reservas": ["RES-0001", "..."] }
```

8. Pre-allocation (Preasignación)

Qué es. Cuando se conoce la estructura de antemano, se crea el documento completo desde el inicio y la aplicación solo rellena los valores.

Cuándo usarlo. Cuando la estructura del documento se sabe de antemano.

Frente al modelo relacional. En relacional se insertarían filas a medida que se necesiten. Aquí el documento ya existe y solo se hacen actualizaciones.

Ventajas y desventajas. A favor: diseño más simple cuando la estructura es conocida. En contra: hay un equilibrio entre simplicidad y rendimiento.

Código P8 · Calendario de disponibilidad preasignado

```
// Calendario mensual con los 30 días libres ("L")
{
  "vehiculo_id": "VEH-0021",
  "mes": "2026-11",
  "dias": { "1": "L", "2": "L", "3": "L" /* ... hasta "30" */ }
}
// Al confirmar una reserva solo se cambia una posición:
db.calendario.updateOne(
  { vehiculo_id: "VEH-0021", mes: "2026-11", "dias.10": "L" },
  { $set: { "dias.10": "RES-1042" } }
);
```

9. Polymorphic (Polimórfico)

Qué es. Documentos que son más parecidos que distintos se guardan en una misma colección aunque no tengan exactamente los mismos campos.

Cuándo usarlo. Cuando hay variaciones entre documentos pero se quiere consultarlos juntos.

Frente al modelo relacional. En relacional habría que usar herencia de tablas o columnas con muchos nulos. Aquí cada documento lleva solo sus campos.

Ventajas y desventajas. A favor: fácil de implementar y consultas sobre una sola colección.

Código P9 · Distintos tipos de vehículo en la colección vehiculos

```
{ "_id": "VEH-0021", "tipo": "automovil", "placa": "KLM123", "puertas": 5, "transmision":
"automática" }
{ "_id": "VEH-0030", "tipo": "moto",      "placa": "ABC12D", "cilindraje": 300,
"casco_incluido": true }
{ "_id": "VEH-0044", "tipo": "camion",    "placa": "XYZ987", "capacidad_kg": 3500 }
```

10. Schema Versioning (Versionado de esquema)

Qué es. Cada documento lleva un campo con la versión de su esquema, y las versiones antiguas y nuevas conviven en la colección mientras se migran.

Cuándo usarlo. En casi cualquier aplicación, porque el esquema cambia durante su vida.

Frente al modelo relacional. En relacional un cambio de esquema suele ser un ALTER TABLE que bloquea o exige ventana de mantenimiento. Aquí no hace falta detener el sistema.

Ventajas y desventajas. A favor: sin tiempo de inactividad, control de la migración y menos deuda técnica. En contra: durante la migración puede hacer falta un índice doble sobre el mismo campo.

Código P10 · Dos versiones de cliente en la misma colección

```
// Versión 1
{ "_id": "CLI-88", "schema_version": 1, "nombre": "Laura Gómez", "telefono": "3001234567" }

// Versión 2: el contacto se agrupa
{ "_id": "CLI-91", "schema_version": 2, "nombre": "Mateo Ruiz",
  "contacto": { "telefono": "3109876543", "email": "mateo@ejemplo.com" } }
```

11. Subset (Subconjunto)

Qué es. En el documento principal se deja solo la parte que se usa con frecuencia y el resto se mueve a otra colección.

Cuándo usarlo. Cuando los documentos son grandes y tienen muchos datos poco usados, de modo que el conjunto de trabajo no cabe en memoria.

Frente al modelo relacional. En relacional se traería todo con JOIN o se paginaría. Aquí la mayoría de las lecturas se resuelven con un solo documento pequeño.

Ventajas y desventajas. A favor: conjunto de trabajo más pequeño y menos acceso a disco. En contra: hay que gestionar el subconjunto y pedir el resto implica otra consulta.

Código P11 · Vehículo con sus reseñas más recientes

```
{
  "_id": "VEH-0021",
  "modelo": "Mazda CX-30",
  "resenas_recientes": [
    { "cliente": "Laura G.", "puntaje": 5, "texto": "Muy limpio y puntual" },
    { "cliente": "Mateo R.", "puntaje": 4, "texto": "Buen consumo" }
  ]
}
// Colección resenas: todas las demás reseñas, referenciadas por vehiculo_id
```

12. Tree (Árbol)

Qué es. Los datos jerárquicos se modelan guardando en cada nodo la referencia a su padre o a sus ancestros.

Cuándo usarlo. Cuando hay jerarquías que se consultan con frecuencia.

Frente al modelo relacional. En relacional se usarían consultas recursivas o varios JOIN. Aquí se consulta un nodo y se obtiene su ruta.

Ventajas y desventajas. A favor: mejor rendimiento al evitar varios JOIN. En contra: las actualizaciones de la jerarquía las debe gestionar la aplicación.

Código P12 · Ubicaciones de SIRAV (país, ciudad, sede)

```
{ "_id": "CO",          "nombre": "Colombia",                "padre": null }
{ "_id": "BOG",         "nombre": "Bogotá",   "padre": "CO",   "ancestros": ["CO"] }
{ "_id": "BOG-CHAPI",  "nombre": "Sede Chapinero", "padre": "BOG", "ancestros": ["CO", "BOG"] }
```

```
// Todas las sedes de Bogotá (cualquier nivel):
db.ubicaciones.find({ ancestros: "BOG" })
```

Guion del video

Duración objetivo: unos 12 minutos y medio, dentro del rango de 12 a 13 minutos que pide el enunciado. Cada patrón dura cerca de 55 segundos y los dos integrantes se alternan. Los tiempos son una guía para ensayar y hay que ajustarlos con el video grabado.

Tiempo	Segmento	Quién	Qué se muestra
0:00	Introducción y presentación del equipo	Ambos	Portada, nombres y por qué importan los patrones en NoSQL
0:45	1. Approximation	David	Definición, cuándo usarlo y el JSON del código P1
1:40	2. Attribute	José David	Definición, cuándo usarlo y el JSON del código P2
2:35	3. Bucket	David	Definición, cuándo usarlo y el JSON del código P3
3:30	4. Computed	José David	Definición, cuándo usarlo y el JSON del código P4
4:25	5. Document Versioning	David	Definición, cuándo usarlo y el JSON del código P5
5:20	6. Extended Reference	José David	Definición, cuándo usarlo y el JSON del código P6
6:15	7. Outlier	David	Definición, cuándo usarlo y el JSON del código P7
7:10	8. Pre-allocation	José David	Definición, cuándo usarlo y el JSON del código P8
8:05	9. Polymorphic	David	Definición, cuándo usarlo y el JSON del código P9
9:00	10. Schema Versioning	José David	Definición, cuándo usarlo y el JSON del código P10
9:55	11. Subset	David	Definición, cuándo usarlo y el JSON del código P11
10:50	12. Tree	José David	Definición, cuándo usarlo y el JSON del código P12
11:45	Cierre y conclusiones	Ambos	Cuadro resumen de los doce patrones

Con estos tiempos el video termina cerca de los 12:30.

Estructura de cada patrón

1. Definición: qué es y cómo funciona en un documento.

2. Contexto de uso: cuándo y por qué usarlo, y la ventaja frente al modelo relacional.
3. Ejemplo práctico: el JSON de SIRAV, resaltando en pantalla el campo clave del patrón.

Recursos visuales sugeridos

- Una diapositiva por patrón con el JSON del ejemplo y una flecha o recuadro que marque lo importante.
- Un diagrama «antes y después» para Extended Reference, Subset y Tree.
- Un corte de pantalla de MongoDB Compass o VS Code con el documento real para dos o tres patrones.

Entregable y lista de verificación

El enunciado pide un único PDF por grupo con el título de la tarea, la asignatura, los nombres completos de los integrantes y el enlace al video.

- Duración entre 12 y 13 minutos.
- Aparecen y hablan todos los integrantes del grupo del proyecto.
- Se cubren los doce patrones del resumen de MongoDB.
- Cada patrón tiene definición, contexto de uso y ejemplo en JSON.
- El video está en YouTube con visibilidad *pública*, no privada ni restringida.
- El PDF incluye título, asignatura, nombres completos y el enlace directo.

Fuentes

1. MongoDB. *Building with Patterns: A Summary*. mongodb.com/company/blog/building-with-patterns-a-summary. Es la referencia que indica el enunciado. La lista de patrones y sus ventajas y desventajas se tomaron de ese resumen, leído a través de una copia publicada en dev.to.

Los ejemplos JSON y las explicaciones de «frente al modelo relacional» son de elaboración propia, a partir de lo visto en clase.